dorodango

Draw squircles in Typst with tunable corner smoothing

Contents

Introduction

`dorodango` provides three functions for drawing rectangles with smoothly rounded corners:

- `squircle`: the [Figma curve family](https://www.figma.com/blog/desperately-seeking-squircles/), transitioning from straight edges into circular arcs via two cubic Bézier shoulders.
- `superellipse`: cubic approximations of Lamé curve ($|\frac{x}{a}|^n + |\frac{y}{b}|^n = 1$) corners, parameterized by an exponent n. For $n > 2$, the ideal curve meets straight edges with zero curvature.
- `clothoid`: cubic approximations of Euler-spiral blend corners, whose ideal curvature ramps linearly along arc length from zero to $\frac{1}{r}$ into a central arc.

Rectangle Squircle Superellipse Clothoid

API reference

squircle

Draws a rectangle with smoothly rounded corners.

Use `squircle` like `rect` when you want softer, more continuous corner transitions. With `smoothing: 0%`, it has the same geometry as a rounded `rect`.

Parameters

```
squircle(
  width: auto length ratio relative ,
  height: auto length ratio relative fraction ,
  fill: none color gradient tiling ,
  stroke: auto none length color gradient tiling stroke dictionary ,
  radius: length ratio relative dictionary ,
  inset: length ratio relative dictionary ,
  outset: length ratio relative dictionary ,
  smoothing: length ratio relative dictionary ,
  preserve-smoothing: bool ,
  per-edge-smoothing: bool ,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The squircle's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The squircle's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the squircle.

When setting a fill, the default stroke disappears. To create a squircle with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the squircle. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top` : The top stroke.
  - `right` : The right stroke.
  - `bottom` : The bottom stroke.
  - `left` : The left stroke.
  - `x` : The left and right stroke.
  - `y` : The top and bottom stroke.
  - `rest` : The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the squircle's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left` : The top-left corner radius.
  - `top-right` : The top-right corner radius.
  - `bottom-right` : The bottom-right corner radius.
  - `bottom-left` : The bottom-left corner radius.
  - `left` : The top-left and bottom-left corner radii.
  - `top` : The top-left and top-right corner radii.
  - `right` : The top-right and bottom-right corner radii.
  - `bottom` : The bottom-left and bottom-right corner radii.
  - `rest` : The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the squircle's content. See `box` 's `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the squircle's size without affecting the layout. See `box` 's `outset` parameter for more details.

Default: `0pt`

smoothing `length` or `ratio` or `relative` or `dictionary`

How strongly to smooth the squircle's corners. Smoothing replaces the ends of each circular corner arc with Bézier transitions whose curvature gradually changes between the straight edges and the arc. This can be:

- A relative length for uniform corner smoothing.
- A dictionary describing the smoothing for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left` : The top-left corner smoothing.
  - `top-right` : The top-right corner smoothing.
  - `bottom-right` : The bottom-right corner smoothing.
  - `bottom-left` : The bottom-left corner smoothing.
  - `left` : The top-left and bottom-left corner smoothing.
  - `top` : The top-left and top-right corner smoothing.
  - `right` : The top-right and bottom-right corner smoothing.
  - `bottom` : The bottom-left and bottom-right corner smoothing.
  - `rest` : The smoothing for all corners except those for which the dictionary explicitly sets smoothing.

At `0%` , the corner is a quarter circle matching a rounded `rect` . At `100%` , it is two Bézier transitions with no circular arc.

A corner's two edges normally share one available space, the smaller of the two. See `per-edge-smoothing` to change that.

Default: `60%`

preserve-smoothing `bool`

Whether to preserve the requested smoothing when it exceeds a corner's available space. This has no effect when the smoothing already fits.

If `false` , smoothing is reduced to fit, so further increases may not change the corner. If `true` , its arc and smoothing angles are retained by compressing the Bézier transition, which can make the corner look compressed.

Default: `false`

per-edge-smoothing `bool`

Whether each half of a corner can use all the space available on its edge when the requested smoothing exceeds the space available along one or both edges.

If `false`, each corner's two transition angles stay symmetric, so if one edge has less room for smoothing, the other half is clamped to that limit and is less smoothed than it could be. If `true`, the two halves can be smoothed independently.

Default: `false`

..body `none` or `content` or `str` or `symbol`

The content to place into the squircle. Strings and symbols are converted to content.

When this is omitted, the squircle takes on a default size of at most `45pt` by `30pt`.

superellipse

Draws a rectangle with superellipse (Lamé curve) corners.

Use `superellipse` like `rect` when you want cubic approximations of Lamé curve corners parameterized by a power exponent. For exponents above 2, the ideal Lamé curve has zero curvature where it meets the straight edges. At `exponent: 2`, the corners are circular arcs matching a rounded `rect`. Higher exponents make the corners squarer.

Parameters

```
superellipse(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  exponent: int float,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The superellipse's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The superellipse's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the superellipse.

When setting a fill, the default stroke disappears. To create a superellipse with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the superellipse. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the superellipse's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the superellipse's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the superellipse's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

exponent `int` or `float`

The Lamé curve exponent n in $|\frac{x}{a}|^n + |\frac{y}{b}|^n = 1$.

Controls the corner shape profile. Values must be finite and are clamped into $[2, 12]$. Below 2 the curve would bulge inward, while above 12 the three-cubic fit degrades and control points can leave the corner's footprint. Near the upper bound the fit trades fidelity for containment. At 2 the curve is a circle, and the corner drawn is the circular arc `rect` draws rather than an approximation.

Default: `5`

..body `none` or `content` or `str` or `symbol`

The content to place into the superellipse. Strings and symbols are converted to content.

When this is omitted, the superellipse takes on a default size of at most `45pt` by `30pt`.

clothoid

Draws a rectangle with clothoid (Euler-spiral) blend corners.

Use `clothoid` like `rect` when you want cubic approximations of Euler-spiral corner transitions, whose ideal curvature ramps linearly along arc length. At `smoothing: 0%`, it has the same geometry as a rounded `rect`.

Parameters

```
clothoid(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  smoothing: length ratio relative dictionary,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The clothoid's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The clothoid's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the clothoid.

When setting a fill, the default stroke disappears. To create a clothoid with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the clothoid. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the clothoid's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the clothoid's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the clothoid's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

smoothing `length` or `ratio` or `relative` or `dictionary`

How strongly to smooth the clothoid's corners. Smoothing splits the 90-degree corner rotation into clothoid spiral transitions (where curvature ramps linearly) and a central circular arc. This can be:

- A relative length for uniform corner smoothing.
- A dictionary describing the smoothing for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner smoothing.
  - `top-right`: The top-right corner smoothing.
  - `bottom-right`: The bottom-right corner smoothing.
  - `bottom-left`: The bottom-left corner smoothing.
  - `left`: The top-left and bottom-left corner smoothing.
  - `top`: The top-left and top-right corner smoothing.
  - `right`: The top-right and bottom-right corner smoothing.
  - `bottom`: The bottom-left and bottom-right corner smoothing.
  - `rest`: The smoothing for all corners except those for which the dictionary explicitly sets smoothing.

At `0%`, the corner is a quarter circle matching a rounded `rect`. At `100%`, it is two clothoid transitions meeting with no circular arc.

When the natural blend does not fit the tighter adjacent-edge budget, its clothoid lengths and effective circular radius scale down together. This keeps the angular smoothing proportions unchanged.

Default: `60%`

..body `none` or `content` or `str` or `symbol`

The content to place into the clothoid. Strings and symbols are converted to content.

When this is omitted, the clothoid takes on a default size of at most `45pt` by `30pt`.

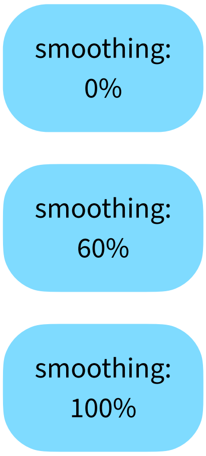
Concepts and examples

Squircle

Smoothing

`smoothing` controls the gradual transition between edges and corners. In the example below, the shapes use `0%`, `60%`, and `100%` smoothing.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(0%, 60%, 100%).map(s => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      smoothing: s,
      fill: aqua,
    )[smoothing: #repr(s)]
  ]),
)
```



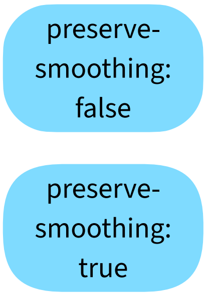
Preserve smoothing

Large radii and high smoothing both consume space along a corner's two edges. When they require more space than is available, `preserve-smoothing` controls whether smoothing is reduced to fit or retained by compressing the Bézier transitions.

- `preserve-smoothing: false` keeps the requested radius within the limits of the shape and lowers smoothing until the corner fits.
- `preserve-smoothing: true` keeps both the requested radius and smoothing, shortening the Bézier transitions between the straight edges and the arc to make the corner fit.

Each corner has available space along each of its two edges, determined by the edge length and the radii of the corners at either end. With radius r and smoothing s, a corner needs $p = (1 + s)r$ along each edge. If p exceeds the available space b, `preserve-smoothing: false` reduces smoothing to $\frac{b}{r} - 1$ while keeping the radius within the limits of the shape. With `preserve-smoothing: true`, the radius and requested smoothing are kept, and the Bézier transitions are shortened to fit.

```
#grid(
  rows: 2,
  gutter: 12pt,
  ..(false, true).map(keep => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 18pt,
      smoothing: 100%,
      preserve-smoothing: keep,
      fill: aqua,
    )[preserve-smoothing: #repr(keep)]
  ]),
)
```



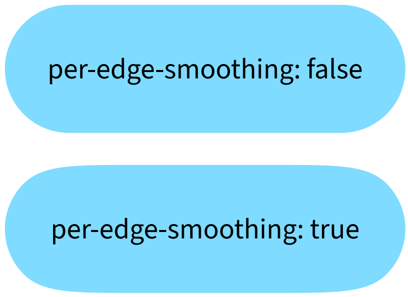
Per-edge smoothing

Each half of a corner uses space along one of its two edges. The edges can have different amounts of available space. For example, on an elongated shape, the short edge may not accommodate the requested smoothing while the long edge still can. `per-edge-smoothing` controls whether the two halves share the tighter limit or use their available space independently.

- `per-edge-smoothing: false` uses the smaller of the two edge budgets for both halves, keeping the corner symmetric. If one edge has less room for smoothing, the other half is also limited to it and is less smoothed than it could be.
- `per-edge-smoothing: true` determines the response to limited space independently for each half.

When `preserve-smoothing` is `false`, smoothing is reduced only for corner halves that do not have enough space for the requested smoothing. Because the two halves can have different amounts of space available, their smoothing may be reduced by different amounts, resulting in different transition angles. When `preserve-smoothing` is `true`, both halves retain the requested smoothing and transition angles. Any half without enough space instead has its Bézier transition compressed to fit.

```
#grid(
  rows: 2,
  gutter: 12pt,
  ..(false, true).map(u => align(center +
horizon)[
    #squircle(
      width: 150pt,
      height: 48pt,
      radius: 25pt,
      smoothing: 100%,
      per-edge-smoothing: u,
      fill: aqua,
    )[per-edge-smoothing: #repr(u)]
  ]),
)
```

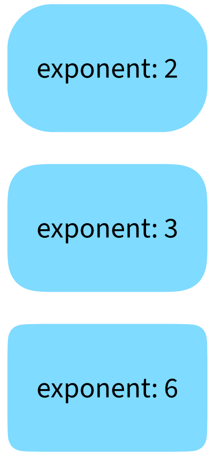


Superellipse

Exponent

The `exponent` parameter on `superellipse` controls how sharp or square the corner profile is. It must be finite and is clamped into $[2, 12]$. At $n = 2$, the curve is a circle, drawing the same circular arc as `rect` rather than an approximation. Near the upper bound the cubic fit trades fidelity for staying inside the corner's footprint.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(2, 3, 6).map(n => align(center + horizon)[
    #superellipse(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      exponent: n,
      fill: aqua,
    )[exponent: #n]
  ]),
)
```



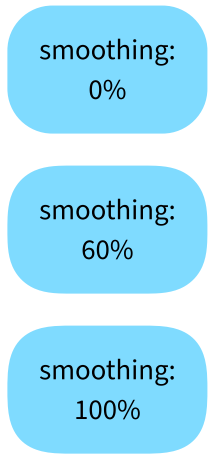
Clothoid

Smoothing

The `clothoid` function approximates Euler spirals to transition into rounded corners. Its ideal profile ramps curvature continuously across every seam.

For positive smoothing, the natural blend can require more space than the tighter adjacent-edge budget allows. In that case, `clothoid` uniformly scales the clothoid lengths and effective circular radius to fit while preserving its angular smoothing proportions. This happens after ordinary radius clamping. At `0%`, the function follows rounded-rectangle geometry instead.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(0%, 60%, 100%).map(s => align(center +
horizon)[
    #clothoid(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      smoothing: s,
      fill: aqua,
    )[smoothing: #repr(s)]
  ]),
)
```

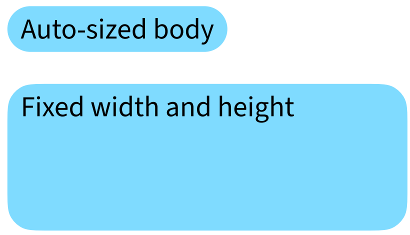


Common parameters

Size and body

`width` and `height` set the squircle's layout size. When both are `auto`, the squircle takes its size from the positional body passed in square brackets, as the first shape below illustrates, while the second uses fixed dimensions.

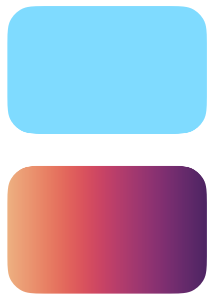
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(radius: 10pt, fill: aqua)[Auto-sized body],
  squircle(
    width: 150pt,
    height: 55pt,
    radius: 10pt,
    fill: aqua,
  )[Fixed width and height],
)
```



Fill

`fill` sets the squircle's interior paint. In the example below, the first shape uses a solid color and the second uses a linear gradient.

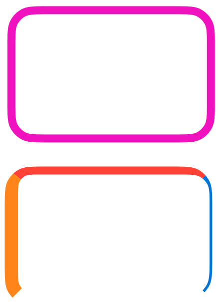
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(width: 75pt, height: 48pt, radius: 10pt, fill: aqua),
  squircle(width: 75pt, height: 48pt, radius: 10pt, fill: gradient.linear(..color.map.flare)),
)
```



Stroke

`stroke` draws the outline. In the example below, the first shape uses one stroke on every edge, while the second assigns each edge its own stroke.

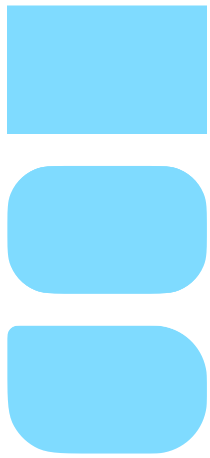
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: 3pt + fuchsia,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: (top: 3pt + red, bottom: none, right: blue, left: 5pt + orange),
  ),
)
```



Radius

`radius` controls corner rounding. In the example below, the shapes show square corners, one shared radius, and a smaller top-left radius with a shared value for the remaining corners.

```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 0pt,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 35%,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: (top-left: 4pt, rest: 20pt),
    fill: aqua,
  ),
)
```



Inset and outset

`inset` pads the body, while `outset` expands the drawing without changing layout. In the example below, the first shape has neither, the second uses an inset, and the third uses an outset.

```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    fill: aqua,
  )[No inset or outset],
  squircle(
    width: 90pt,
    height: 48pt,
    inset: (x: 20pt, y: 10pt),
    radius: 10pt,
    fill: aqua,
  )[Inset adds padding],
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    outset: 8pt,
    fill: aqua,
  )[Outset expands the drawing],
)
```